

阅读报告：MM3DGS SLAM

Anonymous

April 19, 2026

Abstract

本报告针对基于多模态 3D 高斯泼溅（MM3DGS）的 SLAM 系统进行深入分析与评述。首先阐述了传统 SLAM 方法在稀疏地图与密集地图之间的权衡困境，以及现有 3DGS 方法依赖 RGB-D 输入的局限性。在此基础上，介绍了 MM3DGS 采用纯视觉 + 惯性多模态输入、结合 3D 高斯泼溅表示的优势。随后，本文比较了 ORB-SLAM3、DROID-SLAM、GO-SLAM 三种代表性工作，并给出了个人评价。最后简要总结了 MM3DGS 的核心方法。

1 背景与动机

现在很多自主系统（比如无人机、机器人）都面临一个挺头疼的问题：它们要一边建地图，一边在地图里确定自己的位置——这就是 SLAM。问题在于，你想建一个稀疏地图吧，虽然快、能定位，但缺细节（精度低）；要是建密集地图，细节是有了，计算量又大得吓人。一直很难两全。后来 3D 高斯泼溅（3DGS）出来以后，给大家一个新思路：用一堆各向异性的三维高斯分布来表示场景，再加上可微分的“泼溅”光栅化渲染，既能保证渲染质量，又能跑到实时帧率。

但现有的那些基于 3DGS 的 SLAM 方法，比如 SplaTAM，还是只靠 RGB-D 输入。深度传感器一远就噪声大、精度差，价格还贵，性价比很低；纯单目视觉 SLAM 呢，又有尺度模糊的问题。所以为了摆脱这些麻烦，本文提出的 MM3DGS SLAM 就采用了一个多模态的办法，同样用 3D 高斯泼溅来建模环境。这个系统能做到高保真三维重建，同时实时跟踪也挺准。跟以前那些依赖昂贵深度传感器或者要提前算好相机路径的方法不一样，MM3DGS 可以直接拿普通消费设备的未校准数据来跑——这点我觉得挺实用的。

2 相关工作

在 SLAM 领域，MM3DGS 与 ORB-SLAM3（2021）、DROID-SLAM（2022）、GO-SLAM（2023）一样致力于优化 SLAM 流程。下面对这三种方法分别评述，并给出个人看法。

2.1 ORB-SLAM3（Campos et al., IEEE TRO 2021）

ORB-SLAM3 的核心是基于 ORB 特征点的视觉-惯性紧耦合 SLAM，支持多地图、多会话，采用了 IMU 预积分和局部/全局束调整（BA）。与 MM3DGS 相比，其短板在于生成的地图是稀疏点云，无法实现逼真的渲染，且特征点法在低纹理区域容易失效。

个人评价：ORB-SLAM3 是目前最成熟的开源视觉-惯性 SLAM 库，鲁棒性和精度均非常高。但其地图视觉质量差，不能直接用于新视角合成或 AR 内容叠加。MM3DGS 的

目标是“既要精度又要画质”，两者定位不同。不过，MM3DGS 可以借鉴 ORB-SLAM3 的多地图合并思路以及 IMU 偏置估计方法，以提高长时间运行的鲁棒性。

2.2 DROID-SLAM (Teed & Deng, NeurIPS 2022)

DROID-SLAM 的核心思路是将 RAFT 光流网络的迭代更新架构引入 SLAM，加上一个可微分的稠密 BA 层 (DBA)，联合优化相机位姿和每个像素的逆深度。它支持单目、双目和 RGB-D，适用范围较广。与 MM3DGS 相比，DROID-SLAM 的地图表示是深度图而非显式的三维高斯；此外它没有融合 IMU，全局 BA 只能离线运行，无法在线实时调整。

个人评价：DROID-SLAM 在跟踪精度上非常强大，其 DBA 层设计影响了后续许多工作（如 GO-SLAM 和 MM3DGS 的位置优化）。但缺点也很明显：第一，深度图表示无法进行逼真的渲染，不能实现新视角合成；第二，缺乏惯性融合，在快速运动或剧烈旋转时容易丢失跟踪；第三，离线全局 BA 意味着无法在线纠正漂移，长期运行误差会累积。MM3DGS 正好在这些方面做了改进——引入了 3D 高斯泼溅和 IMU 预积分，弥补了 DROID-SLAM 的短板。

2.3 GO-SLAM (Zhang et al., CVPR 2023)

GO-SLAM 在 DROID-SLAM 的基础上增加了在线回环检测和全局 BA，并将地图表示替换为 NeRF 加多分辨率哈希编码，目标是实现隐式表面重建。与 MM3DGS 的不同点在于：GO-SLAM 仍未融合 IMU；NeRF 的渲染速度较慢，大约只能达到 8-10 帧每秒，而 MM3DGS 利用 3D 高斯泼溅，渲染速度可达 90 帧每秒。

个人评价：GO-SLAM 的在线全局优化思路值得肯定，它证明了即使没有 IMU，只要做好回环检测和全局 BA，长轨迹漂移也能被有效抑制。然而，NeRF 这类隐式表示需要大量采样点，实时性天然受限。相比之下，MM3DGS 选择了显式的高斯表示，获得了更高的渲染帧率，在实时应用中优势明显。不过需要指出的是，MM3DGS 目前尚未集成回环检测，这相对于 GO-SLAM 是一个明显的短板。

3 核心方法

3.1 3D 高斯泼溅表示

每个三维高斯 i 由以下参数定义：

- 位置 $\mu_i \in \mathbb{R}^3$ ；
- 协方差矩阵 $\Sigma_i \in \mathbb{R}^{3 \times 3}$ （对称半正定矩阵）；
- 不透明度 $o_i \in [0, 1]$ ；
- 颜色 $\mathbf{c}_i$ （常用球谐系数表示，实现视角相关效果）。

形状参数化：为保证协方差正定，将其分解为旋转和缩放：

$$\Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T,$$

其中 $\mathbf{R}$ 为旋转矩阵， $\mathbf{S}$ 为对角缩放矩阵。优化时只需优化四元数（代替 $\mathbf{R}$ ）和缩放向量。

投影到 2D：3D 高斯投影到图像平面得到 2D 协方差：

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^T \mathbf{J}^T,$$

$\mathbf{W}$ 为世界到相机坐标系的变换矩阵， $\mathbf{J}$ 为投影函数的雅可比矩阵（仿射近似）。

渲染方程：给定相机位姿 $\mathbf{T}_c$ ，像素颜色通过对深度排序后的高斯进行 α 混合得到：

$$C(\mathcal{G}, \mathbf{T}_c) = \sum_{i=1}^N \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j),$$

其中 α_i 由 2D 高斯值乘以不透明度得到。类似地，像素不透明度为：

$$O(\mathcal{G}, \mathbf{T}_c) = \sum_{i=1}^N \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j).$$

优势：渲染过程完全可微，支持通过光度误差反向传播优化高斯参数；相比 NeRF，3DGS 的“泼溅”光栅化效率极高（可达数百 FPS），适合 SLAM 实时应用。

3.2 跟踪损失函数

跟踪阶段固定高斯地图，仅优化相机位姿。损失函数为：

$$L_{\text{tracking}} = \mathbf{1}_{\{O(\mathcal{G}, \mathbf{T}_c) > 0.99\}} (L_{\text{photo}} + \lambda_D L_{\text{depth}})$$

符号说明：

- $\mathcal{G}$ ：当前高斯地图；
- $\mathbf{T}_c$ ：待优化的相机位姿（ $SE(3)$ ）；
- $\lambda_D = 0.05$ （深度损失权重）；
- L_{depth} ：深度相关性损失。

示性函数作用：仅当渲染不透明度 $O > 0.99$ 的像素参与损失计算，避免未建图区域引入梯度噪声。

光度损失： $L_{\text{photo}} = L_1(I, C(\mathcal{G}, \mathbf{T}_c))$ ，L1 范数对异常值更鲁棒。

设计动机：不透明度掩码可防止空洞区域误导优化；IMU 预积分为位姿优化提供初始估计，避免快速运动时陷入局部极小值。

3.3 深度监督（本文核心创新点之一）

单目深度估计（如 DPT）输出相对逆深度，存在尺度模糊。直接使用 L2 损失无法对齐尺度，因此本文采用皮尔逊相关系数作为深度损失：

$$L_{\text{depth}} = \frac{\text{Cov}(D_e, D_r)}{\sqrt{\text{Var}(D_e) \text{Var}(D_r)}}$$

其中 D_e 为估计深度， D_r 为渲染深度。该损失最大化两个深度图的线性相关性，对绝对尺度和偏移不敏感。

实现细节：在初始化新高斯点时，先通过最小二乘求解尺度 σ 和平移 θ ，将估计深度对齐到地图深度：

$$\begin{bmatrix} d_e & \mathbf{1} \end{bmatrix} \begin{bmatrix} \sigma \\ \theta \end{bmatrix} = d_r$$

之后再用皮尔逊损失微调。

个人解析：此设计巧妙利用深度先验提升几何质量，同时避免深度噪声和尺度模糊的破坏。相比 SplaTAM 的加权 L1 深度损失，MM3DGS 的皮尔逊损失更鲁棒。论文图 4 显示，椅子背面的小孔重建细节甚至优于 GT 深度。

3.4 惯性融合

IMU 提供加速度 $\mathbf{a}$ 和角速度 $\dot{\Theta}$ 。两帧之间的相对位姿变化通过预积分计算：

$${}^{t-1}\Delta\mathbf{p}_t = \mathbf{v}_{t-1}\Delta t + \frac{1}{2}\mathbf{a}\Delta t^2, \quad {}^{t-1}\Delta\Theta_t = \dot{\Theta}_{t-1}\Delta t$$

速度递推： $\mathbf{v}_{t-1} = \mathbf{v}_{t-2} + \mathbf{a}_{t-1}\Delta t$ 。由此得到相对变换 ${}^{t-1}\mathbf{T}_I$ ，再结合相机-IMU 外参得到相机帧间相对变换 ${}^{t-1}\mathbf{T}_c$ 。

设计原因：在视觉优化前提供初始位姿估计，比恒定速度模型更准，尤其在快速运动或低帧率时增强鲁棒性。本文采用开环方法，不估计 IMU 偏置，理由是在短时间内偏置误差可被后续视觉优化吸收。

个人解析：开环设计降低了系统复杂度，工程上合理。但在长时无视觉特征（如黑暗走廊）场景下，惯性积分偏置会累积漂移。作者已将此列为未来工作（闭环惯性融合）。整体方案“够用就好”，在大多数有视觉纹理的场景中表现良好。

4 实验分析与深度洞察

4.1 数据集与指标

数据集：UT-MM

这个数据集是作者自己采集的，叫作 UT-MM。里面包含了 RGB、深度、IMU、LiDAR，还有真值轨迹——LiDAR 主要用来提供真值做评估用。一共 8 个场景，名字听起来挺直观的：Ego-centric（自中心运动）、Square（方形轨迹）、Fast-straight（快速直线）等等。场景覆盖了不同类型的运动模式，不算多但该有的基本都有了。

评估指标

用了两个指标：

- **ATE RMSE**（绝对轨迹误差的均方根）：衡量跟踪精度，单位是厘米。数值越小越好。
- **PSNR**（峰值信噪比）：衡量渲染质量，单位是 dB。数值越大越好。

基线方法

只比了一个：SpliTAM。它是基于 RGB-D 输入的 3DGS SLAM 方法，算是目前 3DGS SLAM 里比较有代表性的工作。为什么没比更多？我猜可能是因为 MM3DGS 这种多模态（RGB+IMU）的做法，跟纯 RGB-D 的方法直接比较已经能说明问题了。不过只比一个基线，感觉稍微有点单薄。得到的结果如下图所示：

TABLE I: Multi-modal SLAM results on the UT-MM dataset: ATE RMSE ↓ is in cm and PSNR ↑ is in dB, with SpliTAM is used as a baseline. Best results are in **bold**. Both depth and inertial measurements benefit tracking and image quality.

Method	Avg		Square-1		Ego-centric-1		Ego-drive		Fast-straight	
	ATE	PSNR	ATE	PSNR	ATE	PSNR	ATE	PSNR	ATE	PSNR
SpliTAM (RGB-D)	12.06	22.03	32.86	18.67	4.40	22.78	4.20	20.61	6.78	26.07
Ours (RGB)	39.14	19.73	59.48	16.54	4.09	23.151	67.20	17.51	25.78	21.71
Ours (RGB+IMU)	33.23	19.58	44.26	17.01	3.41	22.96	68.50	17.12	16.78	21.24
Ours (RGB-D)	8.75	22.20	20.38	16.55	6.86	22.24	4.25	23.58	3.52	26.42
Ours (RGB-D+IMU)	3.98	23.30	7.11	18.59	1.15	24.95	4.54	23.61	3.13	26.05

Figure 1: Multi-modal SLAM results on UT-MM dataset.

4.2 关键发现（实验跑出来的几个点）

- 纯 **RGB** 效果最拉胯：ATE 高达 39 cm，基本上没法用。原因也很明显——单目尺度模糊，又没有深度先验，光靠图像根本撑不住。
- 加了 **IMU** 之后，**RGB+IMU** 比纯 **RGB** 有明显提升：ATE 从 39 cm 降到了 33 cm，进步是有的，但跟 RGB-D 比还是差一截。
- **RGB-D+IMU** 组合表现最好：ATE 做到了 3.98 cm，跟 SplaTAM（12.06 cm）相比，提升了差不多 3 倍；PSNR 也涨了 5% 左右。这个提升幅度还是挺可观的。

4.3 我的疑问与洞察（读到这里觉得有点意思的地方）

1. 为什么 **RGB-D+IMU** 相比纯 **RGB-D**，ATE 从 8.75 降到 3.98，但 PSNR 只从 22.20 涨到 23.30？说实话，这个差距有点意思。ATE 降了一半还多，但 PSNR 几乎没怎么动。

我的推测：IMU 主要改善的是轨迹的全局一致性，尤其是水平方向（XY 平面）的漂移。而渲染质量（PSNR）更多依赖于深度和图像的局部匹配——只要每帧的深度图足够准，局部光度误差就已经很小了，IMU 再加进来也帮不了太多。所以 PSNR 提升不大是说得通的。

图 5 也能印证这一点：纯 RGB-D 其实已经把 Z 方向（深度方向）的漂移消掉了，但 XY 方向的漂移还在；加上 IMU 之后，XY 漂移才被真正修正。这正好解释了为什么 ATE 降幅大而 PSNR 几乎没变——因为 PSNR 对 XY 漂移不敏感，只要每帧渲染的图和真值图看起来差不多，PSNR 就上去了。

2. 在 **Ego-drive** 场景中，为什么 **RGB-D+IMU** 的 ATE 反而比纯 **RGB-D** 还高了一点点（4.54 vs 4.25）？这个反直觉了。作者没有解释，我猜可能的原因：

- 这个场景里可能有动态物体（比如行人、车辆），导致视觉优化本身就不太稳，IMU 再给一个错误的初始估计，反而把位姿带偏了。
- 另一种可能是 IMU 遇到了剧烈振动，短时间内加速度或角速度饱和，开环预积分直接算出一个有偏的初值，后续视觉优化也没能完全拉回来。

这其实提醒了一点：惯性融合不是在所有情况下都稳赚不赔。如果 IMU 数据质量差（饱和、振动、未标定），强行融合反而会拖后腿。一个更鲁棒的做法可能是动态调整融合权重——比如根据 IMU 的噪声方差来实时调整协方差，或者在检测到异常时自动降低 IMU 的信任度。作者没提这个，但我觉得值得做。系统架构：MM3DGS SLAM 包含三个阶段：